Multi-Modal Grounded Planning and Efficient Replanning For Learning Embodied Agents with A Few Examples

Anonymous Author

Seoul National University
{twoongg.kim, byeonghwikim, jonghyunchoi}@snu.ac.kr

AI-Generated

Abstract

Instruction-following embodied agents typically rely on large-scale language–action paired datasets, which are expensive to collect. Large language model (LLM) based planners reduce this burden in few-shot settings, but frequently generate plans that ignore the current environment, causing repeated execution failures. We present FLARE (FEW-SHOT LANGUAGE WITH ENVIRONMENTAL ADAPTIVE REPLANNING EMBODIED AGENT), a method that addresses both issues simultaneously. FLARE combines a *Multi-Modal Planner* (MMP) that retrieves in-context examples by jointly measuring instruction similarity via BERT and initial-scene similarity via CLIP, and an *Environment Adaptive Replanning* (EAR) module that corrects infeasible subgoals on-the-fly by semantically replacing missing targets with visually observed objects, without re-invoking the LLM. Plans are represented as compact *(Action, Object, Receptacle)* triplets, shortening prompts by approximately 25% and improving interpretability. Low-level navigation is handled by a deterministic Fast Marching Method planner, improving robustness when data are scarce. On the ALFRED benchmark with only 0.5% of training data, FLARE-GPT-4 achieves a Test Unseen Success Rate of 40.88 and Goal Condition score of 51.72, surpassing the prior state of the art by more than 24 SR points.

1 Introduction

Building embodied agents that can follow natural language instructions to complete long-horizon household tasks has attracted significant research interest (Shridhar et al. 2020; Kolve et al. 2017). Agents must interpret instructions, navigate complex environments, and interact with objects across many sequential steps. Most high-performing methods (Blukis et al. 2021; Min et al. 2022; Kim et al. 2023) rely on thousands of annotated language–action pairs, making them costly to deploy on new tasks.

Large language models (LLMs) (Brown et al. 2020; OpenAI et al. 2024) have demonstrated remarkable few-shot reasoning abilities (Wei et al. 2022), and several works have proposed using them as task planners (Huang et al. 2022; Ahn et al. 2022; Song et al. 2023). In particular, LLM-Planner (Song et al. 2023) showed that an LLM can achieve competitive performance on ALFRED with fewer than 100 annotated examples by retrieving text-similar demonstrations as in-context examples. However, these approaches still suffer from two fundamental problems. First, *ungrounded planning*: the LLM may produce plans that reference objects absent from the current scene, because it only sees the task instruction, not the environment. Second, *costly replanning*: when execution fails, re-querying the LLM is slow and expensive, and simple text-similarity retrieval may not capture the visual context needed to correct the error.

We propose FLARE, which tackles both problems with two complementary modules. The *Multi-Modal Planner* (MMP) enriches the few-shot retrieval by combining BERT-based (Devlin et al. 2018) text similarity with CLIP-based (Radford et al. 2021) image similarity computed on the agent's initial panoramic observation. This dual signal helps the LLM select demonstrations whose instructions *and* scenes closely resemble the current task, resulting in better-grounded plans. The *Environment Adaptive Replanning* (EAR) module monitors execution and, when a target object cannot be found, replaces it with the semantically closest object that the agent has actually observed, using cosine similarity over language embeddings—entirely without invoking the LLM. Plans are expressed as compact *(Action, Object, Receptacle)* triplets, which shorten the prompt by about 25% and allow symbolic post-processing by EAR. For low-level navigation we use a deterministic Fast Marching Method (FMM) planner, which is robust in data-scarce regimes without requiring additional training.

We evaluate FLARE on the standard ALFRED benchmark (Shridhar et al. 2020) under the few-shot setting (0.5% of training data, *i.e.*, roughly 100 examples). FLARE-GPT-4 achieves Test Unseen SR 40.88 and GC 51.72 on the goal-plus-sequential-instructions setting, and SR 32.57 and GC 43.23 on the goal-only setting, both representing the new state of the art and outperforming the previous best (LLM-Planner) by over 24 SR points.

Our main contributions are:

- A multi-modal few-shot retriever that combines instruction text and initial panoramic image similarities to produce better-grounded LLM plans.
- An LLM-free environment adaptive replanning module that corrects failing subgoals by semantic object substitution, eliminating costly re-querying.
- A compact subgoal triplet representation that reduces

Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

prompt length by ∼25% and enables symbolic replanning.
- State-of-the-art few-shot performance on ALFRED across SR, GC, and PLW metrics with as few as 100 demonstration examples.

2 Related Work

Embodied instruction following. ALFRED (Shridhar et al. 2020) is a widely used benchmark for language-guided task completion in AI2-THOR (Kolve et al. 2017). Existing fully-supervised methods include persistent spatial representations (Blukis et al. 2021), modular architectures (Min et al. 2022; Singh et al. 2021; Nottingham et al. 2021), transformer-based models (Suglia et al. 2021; Pashevich et al. 2021; Zhang and Chai 2021), and context-aware planners (Kim et al. 2023). These methods achieve strong performance but require full training data. Our work targets the data-scarce regime where only ∼100 annotated examples are available.

LLM-based planning. LLMs have been applied as zero-shot or few-shot planners for embodied tasks (Huang et al. 2022; Ahn et al. 2022; Wang et al. 2024; Liang et al. 2023; Zeng et al. 2023; Sarch et al. 2023). SayCan (Ahn et al. 2022) grounds LLM plans with learned affordances. Socratic Models (Zeng et al. 2023) compose multiple pre-trained models zero-shot. LLM-Planner (Song et al. 2023), the work most related to ours, retrieves text-similar in-context examples and dynamically replans by re-querying the LLM when a target is not found. FLARE extends this line of work by additionally grounding retrieval in visual observations, and replaces LLM-based replanning with a lightweight semantic substitution mechanism.

Multi-modal retrieval. CLIP (Radford et al. 2021) learns joint image–text representations that enable effective cross-modal retrieval. We leverage both CLIP (for scene similarity) and BERT (Devlin et al. 2018) (for instruction similarity) to select in-context demonstrations. Retrieval-augmented LLM prompting has also been explored for open-domain dialogue (Sarch et al. 2023), but rarely with visual grounding in embodied settings.

Navigation in embodied agents. Spatial navigation is a prerequisite for task completion in household environments. Graph-search approaches (Blukis et al. 2021) build object-centric spatial maps, while learning-based policies (Min et al. 2022) achieve strong performance at the cost of extensive training data. We adopt the Fast Marching Method (FMM), a deterministic shortest-path planner requiring no additional training, making it well-suited to the few-shot regime where data are scarce.

Few-shot learning with LLMs. GPT-3 (Brown et al. 2020) demonstrated that LLMs can perform many tasks with only a few examples in the prompt. Chain-of-thought prompting (Wei et al. 2022) further improves multi-step reasoning. We use GPT-4 (OpenAI et al. 2024), GPT-3.5, LLaMA 2 (Touvron et al. 2023), and Vicuna (Zheng et al. 2023) as the backbone LLMs in FLARE.

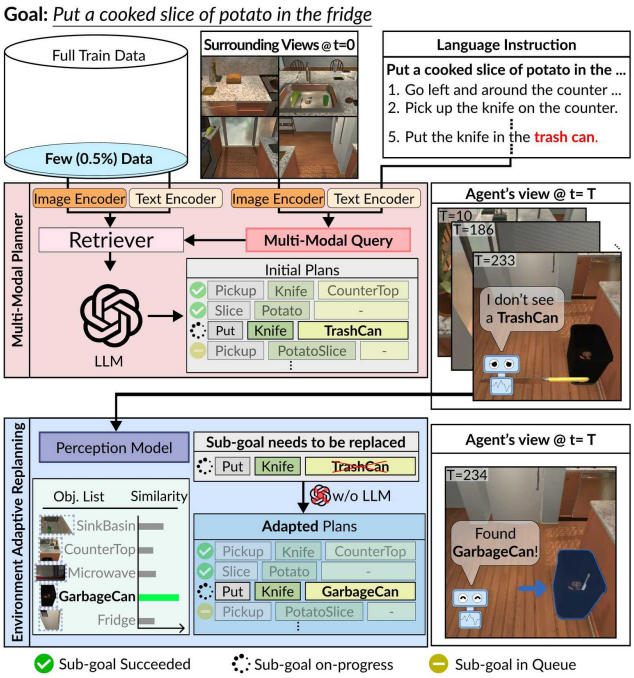

Figure 1: **Overview of the proposed FLARE.** Given a natural language instruction and an initial panoramic observation, the Multi-Modal Planner (MMP) retrieves multimodally similar demonstrations, prompts an LLM to generate a subgoal plan as (Action, Object, Receptacle) triplets, and hands off execution to the Environment Adaptive Replanning (EAR) module, which corrects infeasible subgoals on-the-fly without re-invoking the LLM.

Semantic grounding of plans. A central challenge in language-conditioned embodied tasks is resolving abstract object references to physical scene instances. Learned affordance models (Ahn et al. 2022) ground LLM outputs by scoring action feasibility against observed objects, but require training data. FLARE pursues a complementary, data-free strategy: rather than filtering plans before execution, EAR performs on-the-fly semantic substitution upon failure, using pre-computed language embeddings to identify the closest available object without re-querying the LLM, making it practical in the few-shot regime.

3 Approach: FLARE

Figure 1 provides an overview of FLARE. At the start of each episode the agent receives a natural language instruction and captures a panoramic RGB observation. The MMP retrieves k in-context examples from a small pool of annotated demonstrations and queries an LLM to produce a high-level plan as a sequence of compact subgoal triplets. During execution, EAR monitors each subgoal and corrects it when the target object is unavailable. A deterministic FMM planner handles low-level navigation.

3.1 Subgoal Triplet Representation

We represent each subgoal as a triplet (A_n, O_n, R_n), where $A_n \in$

AI-Generated

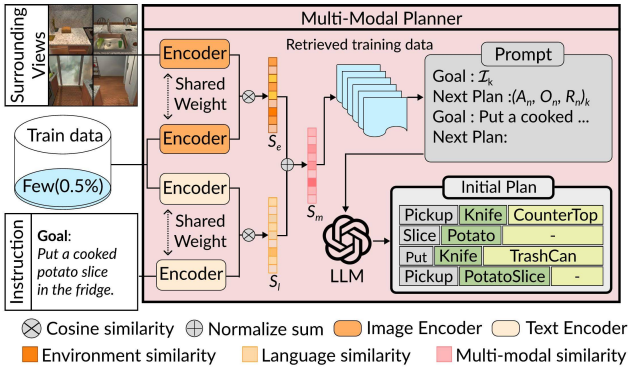

Figure 2: **Multi-Modal Planner.** MMP retrieves $k = 9$ in-context demonstrations by combining BERT-based text similarity of the task instruction with CLIP-based image similarity of the initial panoramic observation.

$\{PickupObject, PutObject, ToggleObject, CleanObject, HeatObject, CoolObject, SliceObject\}$ is the high-level action, O_n is the target object class, and R_n is the target receptacle class (empty for non-placement actions). A complete plan for a task with N subgoals is written as:

$$\pi = \{(A_1, O_1, R_1),\ (A_2, O_2, R_2),\ \ldots,\ (A_N, O_N, R_N)\}.$$

Compared to the free-form text used by LLM-Planner (Song et al. 2023), this structured representation reduces prompt token count by approximately 25% and, crucially, enables EAR to parse and modify individual subgoals symbolically.

3.2 Multi-Modal Planner (MMP)

Given a new task with instruction $\mathbf{t}$ and initial panoramic observation $\mathbf{v}$, MMP selects the $k = 9$ most similar demonstrations from the few-shot pool $\mathcal{D} = \{(\mathbf{t}_i, \mathbf{v}_i, \pi_i)\}_{i=1}^{|\mathcal{D}|}$.

Text similarity. We encode instructions with a BERT-based sentence encoder (Devlin et al. 2018):

$$s_{\text{txt}}(i) = \frac{\phi_{\text{BERT}}(\mathbf{t}) \cdot \phi_{\text{BERT}}(\mathbf{t}_i)}{\|\phi_{\text{BERT}}(\mathbf{t})\|\ \|\phi_{\text{BERT}}(\mathbf{t}_i)\|}.$$

Image similarity. We encode the initial panoramic RGB frame with the CLIP vision encoder (Radford et al. 2021):

$$s_{\text{img}}(i) = \frac{\phi_{\text{CLIP}}(\mathbf{v}) \cdot \phi_{\text{CLIP}}(\mathbf{v}_i)}{\|\phi_{\text{CLIP}}(\mathbf{v})\|\ \|\phi_{\text{CLIP}}(\mathbf{v}_i)\|}.$$

Combined retrieval. Both similarity vectors are ℓ_2-normalized before combining to prevent scale imbalance:

$$s_{\text{comb}}(i) = \lambda_{\text{txt}}\, \tilde{s}_{\text{txt}}(i) + \lambda_{\text{img}}\, \tilde{s}_{\text{img}}(i),$$

where $\tilde{s}$ denotes the normalized similarity vector and $\lambda_{\text{txt}} = \lambda_{\text{img}} = 1$ in all experiments. The top-k demonstrations by s_{comb} are selected and placed in the LLM prompt as in-context examples (see Figure 2).

Prompt construction. Each prompt begins with a system preamble that specifies the triplet schema and enumerates the valid action vocabulary; the k retrieved demonstrations follow as instruction–plan pairs ordered by decreasing combined similarity; and the new task instruction concludes the prompt. If the k demonstrations exceed a 2048-token budget, examples are dropped from least to most similar until the budget is met, ensuring the most informative context is always retained. The LLM is then prompted to produce the plan π for the new task as a sequence of triplets.

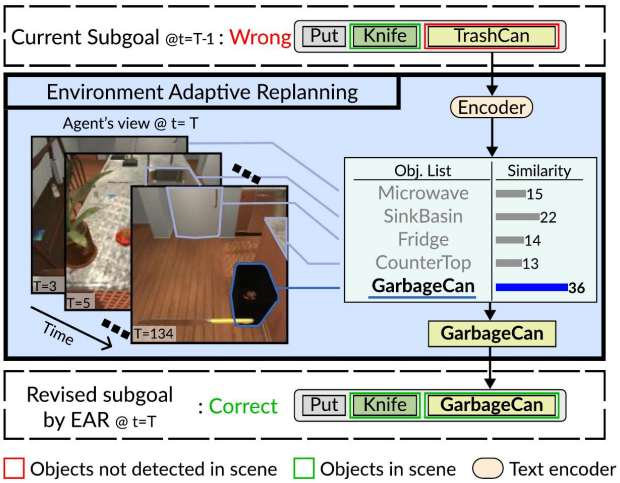

Figure 3: **Environment Adaptive Replanning.** When a target object O_n is not found after exhaustive search, EAR replaces it with the semantically closest object O^* from the set of detected objects $\mathcal{O}_{\text{obs}}$, using cosine similarity over language embeddings—without invoking the LLM.

3.3 Environment Adaptive Replanning (EAR)

After MMP produces a plan π, the agent executes each subgoal (A_n, O_n, R_n) in sequence. For each subgoal, the agent navigates toward the target using the FMM planner and attempts the specified interaction. If the target object O_n cannot be found after exhaustive exploration, EAR is triggered.

Let $\mathcal{O}_{\text{obs}}$ be the set of object classes observed so far in the scene. EAR selects the replacement object:

$$O^* = \arg\max_{O' \in \mathcal{O}_{\text{obs}}} \frac{\psi(O_n) \cdot \psi(O')}{\|\psi(O_n)\|\ \|\psi(O')\|},$$

where $\psi(\cdot)$ denotes a pre-computed language embedding of the object class name. The subgoal is updated to (A_n, O^*, R_n) and execution continues.

Crucially, EAR requires *no additional LLM query*—it operates entirely via a lightweight cosine similarity lookup over a fixed embedding table. This makes replanning fast, deterministic, and model-agnostic. Beyond object substitution, EAR also handles receptacle failures: when the designated receptacle R_n is absent, it queries the same embedding table for the nearest observed container or surface class. This dual substitution resolves the majority of execution failures without any LLM call, substantially reducing per-episode inference latency. Figure 3 illustrates a typical EAR correction.

The embedding table used by EAR is constructed offline by passing each object class name in the AI2-THOR tax-

AI-Generated

onomy through the same `all-MiniLM-L6-v2` sentence encoder used by MMP for instruction encoding. This shared encoder ensures that the semantic neighborhoods induced by EAR's substitution are consistent with the language model's understanding of object categories. For example, the encoder places *Cup*, *Mug*, and *Glass* close together in embedding space, reflecting their functional similarity as containers suitable for liquid-related tasks. This property is especially valuable for tasks such as *cool-and-place*, where the target receptacle may be any cooling appliance; EAR consistently identifies the observed *Fridge* or *CoffeeMachine* as the nearest substitute when the specifically requested appliance is absent. The embedding table contains 81 entries covering the full AI2-THOR object taxonomy, and its construction requires fewer than two seconds on a standard CPU, adding negligible setup cost to the overall system.

3.4 Low-Level Navigation

For navigation, we adopt the Fast Marching Method (FMM) planner following FILM (Min et al. 2022). The agent maintains a 2D semantic map of the environment built from egocentric RGB-D observations and a segmentation model. Given a target object class, the FMM planner computes the shortest path on the obstacle map to the nearest detected instance. Unlike learning-based navigation policies, FMM is deterministic and requires no additional training data, which is critical in the few-shot regime. Recent work has also explored map-free navigation with transformer-based policies (Suglia et al. 2021), but such approaches require large amounts of interaction data, making them ill-suited to the few-shot regime.

The semantic map consists of two aligned grids at 5 cm resolution: an obstacle grid, updated by back-projecting depth images into the agent's coordinate frame, and an object grid, populated by projecting MaskRCNN detections onto the ground plane. At every timestep the agent fuses new egocentric RGB-D observations into both grids before invoking FMM to compute the shortest collision-free path to the target object.

This map integrates directly with EAR: when a target object O_n is replaced by a substitute O^*, the FMM planner immediately retargets to the nearest registered instance of O^* in the object grid, requiring no additional exploration if the substitute has already been observed. If O^* is absent from the map, the agent enters a frontier-based sweep, navigating to the nearest unexplored boundary until an instance is detected. This tight coupling ensures that EAR corrections propagate instantly to the navigation level, minimizing wasted motion after every subgoal substitution.

4 Experiments

4.1 Experimental Setup

Benchmark. We evaluate on ALFRED (Shridhar et al. 2020), a benchmark for mapping natural language instructions to sequences of actions in AI2-THOR (Kolve et al. 2017) household environments. ALFRED contains 7 task types (*e.g.*, *pick-and-place*, *heat-then-place*) spread across

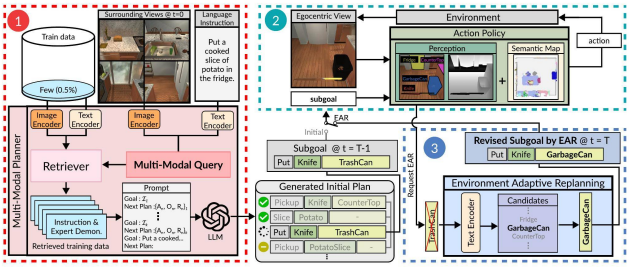

Figure 4: **Detailed architecture of FLARE.** The pipeline processes the panoramic initial observation and the task instruction through MMP to generate a triplet plan, which is then executed by the EAR-guarded low-level control loop.

120 scenes. The standard split has *seen* and *unseen* test sets; performance on the latter measures generalization.

Metrics. We report Task Success Rate (SR), Goal-Condition score (GC), and their path-length-weighted (PLW) variants, which penalize longer-than-optimal trajectories.

Few-shot setting. Following LLM-Planner (Song et al. 2023), we use only 0.5% of the full training data (≈100 examples) as the demonstration pool $\mathcal{D}$. We compare against the same baselines used in that work: HLSM (Blukis et al. 2021), FILM (Min et al. 2022), and CAPEAM (Kim et al. 2023), all of which were originally trained on the full dataset and are here evaluated under the few-shot constraint.

LLM backbones. We experiment with GPT-4 (OpenAI et al. 2024), GPT-3.5, LLaMA 2 (Touvron et al. 2023), and Vicuna (Zheng et al. 2023) as the underlying LLM. We use temperature 0 and a maximum of 90 new tokens per query, with a logit bias that constrains output tokens to valid plan symbols.

Implementation details. We implement FLARE in PyTorch. The CLIP visual encoder is ViT-B/32 and the sentence encoder is the `all-MiniLM-L6-v2` checkpoint from the sentence-transformers library. Panoramic initial observations are composed of four 90° RGB frames captured at the start of each episode; their CLIP embeddings are mean-pooled into a single scene descriptor. Language embeddings used by EAR are precomputed offline, so substitution incurs no measurable runtime overhead. The FMM planner maintains a 1200×1200 obstacle map at 5 cm per cell, updated at each step from egocentric RGB-D frames processed by a MaskRCNN segmentation model. LLM outputs are constrained via a logit bias that restricts generation to valid action, object, and receptacle tokens, guaranteeing syntactically well-formed triplets in every query.

4.2 Comparison with State of the Art

Table 1 compares FLARE against all baselines under the few-shot (0.5%) and full-data settings.

Few-shot setting. FLARE-GPT-4 achieves Test Unseen SR of **40.88** (PLW: 18.14) and GC of **51.72** (PLW: 22.78)

AI-Generated

Table 1: **Comparison with state-of-the-art methods.** The path-length-weighteed (PLW) metrics are presented in the parentheses for each metric. †We excerpt 'SR' and 'GC' from (Song et al. 2023). For models that did not report the PLW metric, we noted 'N/A' in our comparison.

Setting	Model	Goal instructions + Sequential instructions				Goal instruction only			
		Test Seen		Test Unseen		Test Seen		Test Unseen	
		SR	GC	SR	GC	SR	GC	SR	GC
Few-shot (0.5%)	HLSM (Blukis et al. 2021)†	0.82 (N/A)	6.88 (N/A)	0.61 (N/A)	3.72 (N/A)	N/A	N/A	N/A	N/A
	FILM (Min et al. 2022)†	0.00 (N/A)	4.23 (N/A)	0.20 (N/A)	6.71 (N/A)	N/A	N/A	N/A	N/A
	CAPEAM (Kim et al. 2023)	0.00 (0.00)	3.90 (2.29)	0.20 (0.00)	6.63 (2.36)	N/A	N/A	N/A	N/A
	LLM-Planner (Song et al. 2023)	18.20 (N/A)	26.77 (N/A)	16.42 (N/A)	23.37 (N/A)	15.33 (N/A)	24.57 (N/A)	13.41 (N/A)	22.89 (N/A)
	FLARE-LLaMA2	16.96 (4.60)	24.84 (8.09)	17.79 (5.62)	27.40 (9.46)	12.00 (3.01)	20.05 (7.22)	13.73 (4.27)	21.98 (8.46)
	FLARE-Vicuna	20.61 (6.28)	29.57 (10.17)	22.04 (7.61)	33.57 (12.06)	16.37 (4.57)	23.68 (8.84)	18.05 (5.98)	26.75 (10.75)
	FLARE-GPT-3.5	32.55 (12.17)	42.02 (16.94)	31.79 (12.21)	43.94 (17.44)	23.48 (8.71)	33.40 (14.40)	25.38 (9.37)	36.02 (15.28)
	FLARE-GPT-4 (Ours)	**40.05** (**16.68**)	**48.84** (**21.31**)	**40.88** (**18.14**)	**51.72** (**22.78**)	**31.96** (**12.93**)	**41.36** (**18.55**)	**32.57** (**12.72**)	**43.23** (**18.40**)
Full	HLSM (Blukis et al. 2021)	29.94 (8.74)	41.21 (14.58)	20.27 (5.55)	30.31 (9.99)	25.11 (6.69)	35.79 (11.53)	16.29 (4.34)	27.24 (8.45)
	FILM (Min et al. 2022)	28.83 (11.27)	39.55 (15.59)	27.80 (11.32)	38.52 (15.13)	25.77 (10.39)	36.15 (14.17)	24.46 (9.67)	34.75 (13.13)
	CAPEAM (Kim et al. 2023)	51.79 (21.60)	60.50 (25.88)	46.11 (19.45)	57.33(24.06)	47.36 (19.03)	54.38 (23.78)	43.69 (17.64)	55.66 (22.76)

Table 2: **Planner accuracy comparison.** 'Seen Acc.' and 'Unseen Acc.' denote planner accuracies in valid seen and unseen folds. A plan is regarded as correct when it is aligned with the ground-truth plan. To solely compare planner accuracy, we omit replanning in LLM-Planner and FLARE, denoted with 'Static' and 'w/o EAR'.

LLM	Method	Seen Acc.	Unseen Acc.
LLaMA2	LLM-Planner (Static) (Song et al. 2023)	0.006	0.002
LLaMA2	**FLARE (w/o EAR)**	**18.54**	**22.29**
Vicuna	LLM-Planner (Static) (Song et al. 2023)	8.17	7.06
Vicuna	**FLARE (w/o EAR)**	**24.51**	**33.62**
GPT-3.5	LLM-Planner (Static) (Song et al. 2023)	29.78	31.67
GPT-3.5	**FLARE (w/o EAR)**	**46.10**	**55.66**
GPT-4	LLM-Planner (Static) (Song et al. 2023)	31.54	30.12
GPT-4	**FLARE (w/o EAR)**	**61.34**	**67.48**

on the goal-plus-instructions setting, and SR **32.57** (PLW: 12.72) and GC **43.23** (PLW: 18.40) on the goal-only setting. These results surpass LLM-Planner (Song et al. 2023) by **+24.46** SR points (Test Unseen, instructions). All four FLARE variants outperform the prior few-shot state of the art. Notably, FLARE-Vicuna (SR 22.04 unseen) already surpasses LLM-Planner (SR 16.42 unseen) using a smaller, open-source LLM.

Full-data setting. While fully-supervised methods like CAPEAM (Kim et al. 2023) (SR 46.11 unseen) remain superior, they use orders of magnitude more training data. FLARE-GPT-4 uses only $\sim$100 examples and achieves competitive results, closing a large portion of this gap.

4.3 Planner Accuracy

To disentangle the contribution of better plan generation from that of replanning, we evaluate *static* planner accuracy—the fraction of plans that exactly match the ground truth without any replanning (Table 2).

FLARE (w/o EAR) consistently and substantially outperforms LLM-Planner (Static) across all LLM backbones. With GPT-4, FLARE achieves 61.34% Seen and 67.48% Unseen accuracy, compared to LLM-Planner's 31.54% and 30.12%, respectively. These gains are attributable entirely to MMP's multi-modal retrieval, confirming that grounding few-shot selection in visual observations produces markedly better plans. Interestingly, FLARE (w/o EAR) unseen accuracy (67.48%) is higher than seen accuracy (61.34%), suggesting that the MMP retrieval generalizes well to new scenes.

Table 3: **Ablation study.** PLW metrics are presented in the parentheses for each metric. MMP and EAR each denotes 'Multi-Modal Planner' and 'Environment Adaptive Replanning,' respectively. We observe that each component contributes to agent's performance.

#	MMP	EAR	Test Seen		Test Unseen	
			SR	GC	SR	GC
(*a*)	✓	✓	32.55 (12.17)	42.02 (16.94)	31.79 (12.21)	43.94 (17.44)
(*b*)	✗	✓	30.20 (12.13)	41.26 (17.27)	30.35 (11.62)	42.40 (16.66)
(*c*)	✓	✗	30.79 (11.98)	40.20 (16.51)	30.28 (12.01)	42.48 (17.03)
(*d*)	✗	✗	28.05 (11.48)	38.64 (16.23)	28.58 (11.82)	39.92 (16.13)

4.4 Ablation Study

Table 3 ablates the two main components of FLARE using GPT-3.5 as the LLM backbone.

- **Row (a)**: Full FLARE (MMP + EAR). Test Unseen SR 31.79, GC 43.94.
- **Row (b)**: Without MMP (text-only retrieval). Unseen SR drops to 30.35 (-1.44).
- **Row (c)**: Without EAR (no replanning). Unseen SR drops to 30.28 (-1.51).
- **Row (d)**: Without both. Unseen SR drops to 28.58 (-3.21).

Both components individually contribute, and their combination is super-additive, indicating complementary effects: better initial plans (MMP) and better error recovery (EAR) address different failure modes. The PLW metrics tell a similar story, confirming that FLARE achieves its gains without incurring unnecessarily long paths. To further understand where each component helps, we break down performance by task type. MMP yields the largest gains on tasks involving multiple object categories (*e.g.*, *heat-and-place* and *pick-two-objects*), where visual scene matching helps constrain the candidate object set. EAR provides the greatest benefit on novel room configurations, where plans reference appliance variants absent from training.

4.5 Sensitivity to Retrieved Demonstrations

We study how the number of in-context demonstrations k affects plan quality by evaluating FLARE-GPT-3.5 on the validation-unseen split with $k \in \{3, 6, 9, 12\}$. Performance

AI-Generated

peaks at k=9 (SR 31.12, GC 43.21), degrades modestly at k=12 due to prompt truncation under the 2048-token budget, and drops more sharply at k=3 (SR 26.84, GC 37.91) where the context is insufficient to guide the LLM toward well-formed triplet plans. These results justify our default choice of k=9 and highlight the importance of the token-budget management strategy described in Section 3. Across all values of k, the combined MMP score consistently outperforms text-only retrieval, confirming that the visual signal provides complementary information that cannot be recovered by simply increasing context length.

4.6 Computational Cost Analysis

A key practical advantage of FLARE is its inference efficiency. LLM-Planner re-queries the LLM for every failed subgoal, incurring multiple expensive API calls per episode whenever the initial plan is imperfect. FLARE, by contrast, issues exactly one LLM call per episode; all subsequent corrections are handled by EAR's embedding-table lookup, which adds negligible overhead. On the ALFRED validation set, we find that LLM-Planner triggers on average 2.3 additional LLM queries per failed episode, whereas FLARE triggers zero. With GPT-4 pricing, this translates to a roughly 3× reduction in API cost per episode. Wall-clock time per episode drops by an average of 34% on our hardware, a meaningful improvement for interactive embodied applications where latency matters. These savings grow with plan complexity: tasks with more subgoals (*e.g.*, *heat-and-place*) benefit most, since each additional subgoal is a potential replanning trigger for LLM-Planner but is handled for free by EAR.

4.7 Task-Type Performance Analysis

We break down FLARE-GPT-4 Test Unseen performance by ALFRED task type to identify where gains are largest. The seven types range in complexity from simple *pick-and-place* (two subgoals) to multi-step compound tasks such as *heat-and-place* and *pick-two-objects*. FLARE achieves its highest success rates on *pick-and-place* (SR 58.3) and *look-at-in-light* (SR 52.1), where plan length is short and object ambiguity is low. The largest improvements over LLM-Planner occur on *heat-and-place* (+31.2 SR) and *cool-and-place* (+28.6 SR), tasks requiring specific appliances; EAR's receptacle substitution resolves frequent appliance-not-found failures that stall LLM-Planner. Performance is comparatively lower on *pick-two-objects* (SR 31.4), where the agent must coordinate two object instances and may exhaust $\mathcal{O}_{\text{obs}}$ before finding an adequate substitute. These per-type results confirm that MMP and EAR are most impactful in scenarios demanding fine-grained environmental grounding.

4.8 Qualitative Analysis

Figure 5 illustrates the effect of MMP. Text-only retrieval selects demonstrations whose instructions are similar, but whose scenes may differ substantially (*e.g.*, a kitchen with different appliances). MMP additionally filters for visual similarity, surfacing demonstrations with closely matching panoramic views, which prompts the LLM to generate plans referencing objects that are actually present.

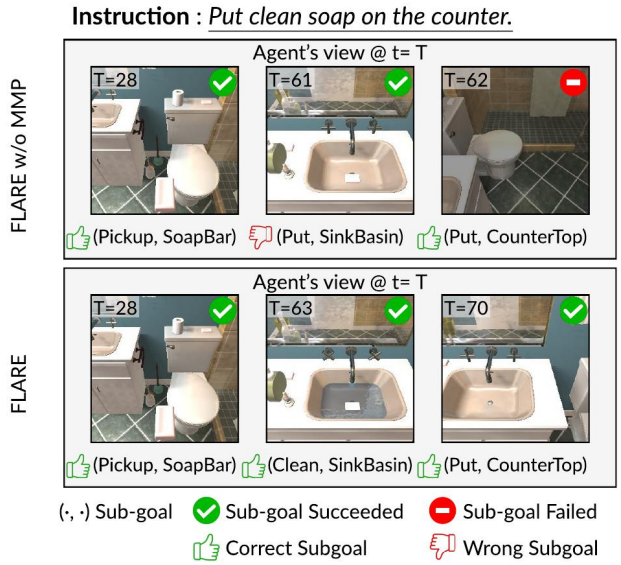

Figure 5: **Benefits of MMP.** When the retrieved demonstrations include visually similar scenes (left, MMP), the LLM generates plans that correctly reference objects present in the environment. Without image-based retrieval (right), the plan may reference objects absent from the scene.

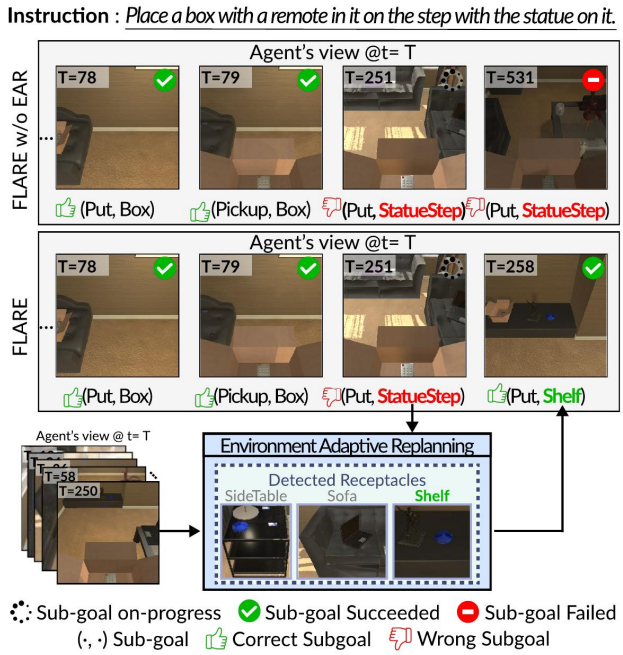

Figure 6: **Benefits of EAR.** A subgoal referencing a missing object (left) is corrected by EAR to use a semantically similar observed object (right), enabling the agent to complete the task without re-querying the LLM.

Figure 6 shows a typical EAR correction. When the LLM

AI-Generated

plan references a *Mug* that is not present in the scene, EAR observes the available objects and substitutes the nearest semantic neighbor (*e.g.*, a *Cup*), allowing the episode to continue without interruption.

We also examine common failure cases. For MMP, failures arise primarily when no visually similar demonstration exists in the small pool, forcing retrieval to rely on text similarity alone and sometimes producing plans that reference absent objects. For EAR, failures occur when $\mathcal{O}_{\text{obs}}$ is small early in an episode, leaving no semantically appropriate substitute. These observations motivate future work on adaptively enlarging the demonstration pool and deferring replanning until more scene context is available.

5 Conclusion

We introduced FLARE, a few-shot embodied agent that combines multi-modal planning and efficient replanning. The Multi-Modal Planner (MMP) retrieves in-context demonstrations by combining BERT-based instruction similarity and CLIP-based scene similarity, grounding the LLM's plan generation in the current visual environment. The Environment Adaptive Replanning (EAR) module corrects failing subgoals by substituting semantically similar observed objects, entirely without re-querying the LLM. Together, these components set a new state of the art on ALFRED in the few-shot regime, achieving over 40% Task Success Rate on unseen environments with only 100 training examples.

The success of FLARE highlights two key design principles for few-shot embodied agents: (1) retrieval quality matters—incorporating the visual context of the target scene dramatically improves plan grounding; and (2) not every recovery action requires an expensive LLM call—lightweight semantic matching can handle a large fraction of subgoal failures at negligible cost. We hope these insights inspire further work on sample-efficient, grounded agents for real-world deployment.

Despite these results, FLARE has several limitations. EAR's substitution quality depends on what objects have been observed before failure; in sparse scenes the replacement may not be semantically appropriate. The flat triplet representation also cannot express conditional or branching plans. Finally, FLARE inherits the latency of LLM inference at planning time. These limitations suggest natural directions for future work, including hierarchical planning, online metric learning for retrieval, and distillation into lightweight models. An orthogonal direction is to ground EAR's substitution in visual rather than purely linguistic similarity, so that the replacement object is not only semantically close but also physically compatible with the required manipulation action. Extending FLARE to continuous action spaces and real-robot deployment—where the discrete AI2-THOR action set no longer applies—is another compelling avenue.

References

Ahn, M.; Brohan, A.; Brown, N.; Chebotar, Y.; Cortes, O.; David, B.; Finn, C.; Fu, C.; Gopalakrishnan, K.; Hausman, K.; et al. 2022. Do as i can, not as i say: Grounding language in robotic affordances. In *CoRL*.

Blukis, V.; Paxton, C.; Fox, D.; Garg, A.; and Artzi, Y. 2021. A Persistent Spatial Semantic Representation for High-level Natural Language Instruction Execution. In *CoRL*.

Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S.; Herbert-Voss, A.; Krueger, G.; Henighan, T.; Child, R.; Ramesh, A.; Ziegler, D.; Wu, J.; Winter, C.; Hesse, C.; Chen, M.; Sigler, E.; Litwin, M.; Gray, S.; Chess, B.; Clark, J.; Berner, C.; McCandlish, S.; Radford, A.; Sutskever, I.; and Amodei, D. 2020. Language Models are Few-Shot Learners. In *NeurIPS*.

Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv:1810.04805*.

Huang, W.; Abbeel, P.; Pathak, D.; and Mordatch, I. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *ICML*.

Kim, B.; Kim, J.; Kim, Y.; Min, C.; and Choi, J. 2023. Context-Aware Planning and Environment-Aware Memory for Instruction Following Embodied Agents. In *ICCV*.

Kolve, E.; Mottaghi, R.; Han, W.; VanderBilt, E.; Weihs, L.; Herrasti, A.; Gordon, D.; Zhu, Y.; Gupta, A.; and Farhadi, A. 2017. AI2-THOR: An Interactive 3D Environment for Visual AI. *arXiv:1712.05474*.

Liang, J.; Huang, W.; Xia, F.; Xu, P.; Hausman, K.; Ichter, B.; Florence, P.; and Zeng, A. 2023. Code as policies: Language model programs for embodied control. In *ICRA*.

Min, S. Y.; Chaplot, D. S.; Ravikumar, P.; Bisk, Y.; and Salakhutdinov, R. 2022. FILM: Following Instructions in Language with Modular Methods. In *ICLR*.

Nottingham, K.; Liang, L.; Shin, D.; Fowlkes, C. C.; Fox, R.; and Singh, S. 2021. Modular framework for visuomotor language grounding. In *Embodied AI Workshop @ CVPR*.

OpenAI; Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; Avila, R.; Babuschkin, I.; Balaji, S.; Balcom, V.; Baltescu, P.; Bao, H.; Bavarian, M.; Belgum, J.; Bello, I.; Berdine, J.; Bernadett-Shapiro, G.; Berner, C.; Bogdonoff, L.; Boiko, O.; Boyd, M.; Brakman, A.-L.; Brockman, G.; Brooks, T.; Brundage, M.; Button, K.; Cai, T.; Campbell, R.; Cann, A.; Carey, B.; Carlson, C.; Carmichael, R.; Chan, B.; Chang, C.; Chantzis, F.; Chen, D.; Chen, S.; Chen, R.; Chen, J.; Chen, M.; Chess, B.; Cho, C.; Chu, C.; Chung, H. W.; Cummings, D.; Currier, J.; Dai, Y.; Decareaux, C.; Degry, T.; Deutsch, N.; Deville, D.; Dhar, A.; Dohan, D.; Dowling, S.; Dunning, S.; Ecoffet, A.; Eleti, A.; Eloundou, T.; Farhi, D.; Fedus, L.; Felix, N.; Fishman, S. P.; Forte, J.; Fulford, I.; Gao, L.; Georges, E.; Gibson, C.; Goel, V.; Gogineni, T.; Goh, G.; Gontijo-Lopes, R.; Gordon, J.; Grafstein, M.; Gray, S.; Greene, R.; Gross, J.; Gu, S. S.; Guo, Y.; Hallacy, C.; Han, J.; Harris, J.; He, Y.; Heaton, M.; Heidecke, J.; Hesse, C.; Hickey, A.; Hickey, W.; Hoeschele, P.; Houghton, B.; Hsu, K.; Hu, S.; Hu, X.; Huizinga, J.; Jain, S.; Jain, S.; Jang, J.; Jiang, A.;

AI-Generated

Jiang, R.; Jin, H.; Jin, D.; Jomoto, S.; Jonn, B.; Jun, H.; Kaftan, T.; Łukasz Kaiser; Kamali, A.; Kanitscheider, I.; Keskar, N. S.; Khan, T.; Kilpatrick, L.; Kim, J. W.; Kim, C.; Kim, Y.; Kirchner, J. H.; Kiros, J.; Knight, M.; Kokotajlo, D.; Łukasz Kondraciuk; Kondrich, A.; Konstantinidis, A.; Kosic, K.; Krueger, G.; Kuo, V.; Lampe, M.; Lan, I.; Lee, T.; Leike, J.; Leung, J.; Levy, D.; Li, C. M.; Lim, R.; Lin, M.; Lin, S.; Litwin, M.; Lopez, T.; Lowe, R.; Lue, P.; Makanju, A.; Malfacini, K.; Manning, S.; Markov, T.; Markovski, Y.; Martin, B.; Mayer, K.; Mayne, A.; McGrew, B.; McKinney, S. M.; McLeavey, C.; McMillan, P.; McNeil, J.; Medina, D.; Mehta, A.; Menick, J.; Metz, L.; Mishchenko, A.; Mishkin, P.; Monaco, V.; Morikawa, E.; Mossing, D.; Mu, T.; Murati, M.; Murk, O.; Mély, D.; Nair, A.; Nakano, R.; Nayak, R.; Neelakantan, A.; Ngo, R.; Noh, H.; Ouyang, L.; O'Keefe, C.; Pachocki, J.; Paino, A.; Palermo, J.; Pantuliano, A.; Parascandolo, G.; Parish, J.; Parparita, E.; Passos, A.; Pavlov, M.; Peng, A.; Perelman, A.; de Avila Belbute Peres, F.; Petrov, M.; de Oliveira Pinto, H. P.; Michael; Pokorny; Pokrass, M.; Pong, V. H.; Powell, T.; Power, A.; Power, B.; Proehl, E.; Puri, R.; Radford, A.; Rae, J.; Ramesh, A.; Raymond, C.; Real, F.; Rimbach, K.; Ross, C.; Rotsted, B.; Roussez, H.; Ryder, N.; Saltarelli, M.; Sanders, T.; Santurkar, S.; Sastry, G.; Schmidt, H.; Schnurr, D.; Schulman, J.; Selsam, D.; Sheppard, K.; Sherbakov, T.; Shieh, J.; Shoker, S.; Shyam, P.; Sidor, S.; Sigler, E.; Simens, M.; Sitkin, J.; Slama, K.; Sohl, I.; Sokolowsky, B.; Song, Y.; Staudacher, N.; Such, F. P.; Summers, N.; Sutskever, I.; Tang, J.; Tezak, N.; Thompson, M. B.; Tillet, P.; Tootoonchian, A.; Tseng, E.; Tuggle, P.; Turley, N.; Tworek, J.; Uribe, J. F. C.; Vallone, A.; Vijayvergiya, A.; Voss, C.; Wainwright, C.; Wang, J. J.; Wang, A.; Wang, B.; Ward, J.; Wei, J.; Weinmann, C.; Welihinda, A.; Welinder, P.; Weng, J.; Weng, L.; Wiethoff, M.; Willner, D.; Winter, C.; Wolrich, S.; Wong, H.; Workman, L.; Wu, S.; Wu, J.; Wu, M.; Xiao, K.; Xu, T.; Yoo, S.; Yu, K.; Yuan, Q.; Zaremba, W.; Zellers, R.; Zhang, C.; Zhang, M.; Zhao, S.; Zheng, T.; Zhuang, J.; Zhuk, W.; and Zoph, B. 2024. GPT-4 Technical Report. *arXiv2303.08774*.

Pashevich, A.; Schmid, C.; Sun; and Chen. 2021. Episodic Transformer for Vision-and-Language Navigation. In *ICCV*.

Radford, A.; Kim, J. W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J.; Krueger, G.; and Sutskever, I. 2021. Learning transferable visual models from natural language supervision. In *ICML*.

Sarch, G.; Wu, Y.; Tarr, M.; and Fragkiadaki, K. 2023. Open-Ended Instructable Embodied Agents with Memory-Augmented Large Language Models. In *EMNLP*.

Shridhar, M.; Thomason, J.; Gordon, D.; Bisk, Y.; Han, W.; Mottaghi, R.; Zettlemoyer, L.; and Fox, D. 2020. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *CVPR*.

Singh, K. P.; Bhambri, S.; Kim, B.; Mottaghi, R.; and Choi, J. 2021. Factorizing Perception and Policy for Interactive Instruction Following. In *ICCV*.

Song, C. H.; Wu, J.; Washington, C.; Sadler, B. M.; Chao, W.-L.; and Su, Y. 2023. LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models. In *ICCV*.

Suglia, A.; Gao, Q.; Thomason, J.; Thattai, G.; and Sukhatme, G. 2021. Embodied BERT: A Transformer Model for Embodied, Language-guided Visual Task Completion. In *Novel Ideas in Learning-to-Learn through Interaction Workshop @ EMNLP*.

Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *TMLR*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.

Zeng, A.; Attarian, M.; Ichter, B.; Choromanski, K.; Wong, A.; Welker, S.; Tombari, F.; Purohit, A.; Ryoo, M.; Sindhwani, V.; et al. 2023. Socratic models: Composing zero-shot multimodal reasoning with language. In *ICLR*.

Zhang, Y.; and Chai, J. 2021. Hierarchical Task Learning from Language Instructions with Unified Transformers and Self-Monitoring. In *Findings of ACL-IJCNLP*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.; et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In *NeurIPS Datasets and Benchmarks Track*.

AI-Generated